PROACTIVE THERMAL MANAGEMENT

MISD: Instruction-Density Driven Thermal Prediction for Edge AI

Using instruction-class profiling to predict thermal spikes
3–5 seconds before they happen

SECTION 1

Status of Thermal Management in Edge Devices

Default Thermal Behavior Under Edge AI Workloads

The Status Quo: The standard Linux thermal_core subsystem and step_wise governor rely on temperature polling intervals of 10–100 ms.

A Reactive System: This architecture is fundamentally reactive—it reads a sensor and applies mitigation only after the thermal event has begun.

The SLM Challenge: While traditional bursty workloads allow for passive cooling, Small Language Model (SLM) inference creates a sustained, plateau-like thermal profile that eliminates idle gaps.

The Result: The device quickly hits its thermal limit (~60 seconds), the OS panic-throttles the CPU frequency (e.g., dropping to 600 MHz), and workload throughput collapses.

Current Approaches to Thermal Control

B0: Default Linux Governor

Uses Digital Thermal Sensor (DTS) temperature polling to throttle the CPU reactively when a trip point is exceeded.

B1: DTS-Only Predictive

Uses a PID controller to compute the rate of temperature change (dT/dt) and extrapolate future temperatures using the existing DTS.

B2: AMU-Only (Activity Monitor)

Relies on the ARM Activity Monitors Unit to track aggregate core activity ratios, triggering throttling when CPU is highly utilized.

Limitations of Existing Hardware Monitors

The Problem with DTS

Digital Thermal Sensors are inherently lagging indicators. They report what has already happened to the silicon, and rapid thermal changes are smoothed out by the thermal mass of the die itself.

The Problem with AMU

Activity Monitors are coarse indicators. They report that a core is busy, but cannot distinguish between thermally benign instructions (e.g., integer ADD) and thermally dangerous ones (e.g., FMLA vector math)—both show 100% utilization.

SECTION 2

Our Hypothesis & Core Concept

The Core Problem Statement

The Critical Latency Gap

There is a severe mismatch between microsecond-scale hardware thermal events and the 10–100 ms polling interval of the OS.

The Key Distinction

Sustained matrix-heavy SLM inference breaks the assumptions of traditional thermal governors built for bursty workloads. This is the strongest element of our problem statement—highlighting how continuous compute eliminates the idle gaps that reactive systems depend on.

The Physics of Dynamic Power (Our Foundation)

$$P(dyn) = \alpha \cdot C \cdot V^2 \cdot f$$

Established Physics: The dynamic power equation is foundational (Chandrakasan et al., 1992).

The Key Insight: Voltage (V) and frequency (f) are visible to and controlled by the OS. Capacitance (C) is fixed by the silicon.

The Hidden Variable: The Activity Factor (α)—the fraction of transistors switching per cycle—is entirely dependent on the specific instructions executing, making it invisible to the OS.

Our Novelty: We are not changing the formula; we are using instruction-class profiling to estimate α in real-time. This provides a software-level proxy for a silicon-level quantity.

What is MISD? (The Elevator Pitch)

The Metric

MISD (Mathematical Instruction Set Density) is a ratio tracking what fraction of instructions in a 10 ms window are power-hungry (NEON vector ops) versus cheap operations (integer ADD).

$$MISD = (ASE_SPEC + VFP_SPEC) / INST_RETIRED$$

A Causal Leading Indicator

Because high-energy instructions generate heat, MISD captures the cause of the thermal spike. This allows us to predict a dangerous temperature rise 3–5 seconds before the temperature sensor detects it.

Why We Don’t Need the Exact Instructions

The Restaurant Kitchen Analogy

To know if a kitchen will get hot, you don’t need the exact recipe for every dish; you just need to know the ratio of orders requiring the grill/oven (high heat) versus salads (no heat).

Aggregate Abstraction

SLM instruction streams shift rapidly. We do not track individual instructions. We only track the aggregate ratio of wide, power-hungry NEON datapath activations. The ratio alone is enough to predict the thermal trajectory.

Is MISD Novel Against Existing Monitors?

Our Hypothesis

MISD is novel because it introduces instruction-class energy weighting as a predictive proxy for α, effectively tracking thermal density before the heat materializes.

Proving Novelty

To prove this is not a repackaging of existing monitors, our objective is to explicitly benchmark MISD against DTS-only and AMU-only baselines. We will demonstrate that understanding what the core is doing (MISD) is better than just knowing how busy it is (AMU).

SECTION 3

What Are We Building?

Part 1: Firestarter Calibration Kernels

Writing tight loops in C with inline ARM assembly to run single-variable thermal stress tests.

Tier 1: HOT

100% FMLA loop

Max compute, highest α

Tier 2: WARM

100% FADD loop

Moderate α

Tier 3: COOL

100% MOV / DUP loop

Data movement

Tier 4: COLD

100% Integer ADD

Baseline scalar

Empirical Validation

We will measure the temperature rise (ΔT) for each tier to empirically prove that instruction mix dictates heat generation.

Part 2: Linear Regression Predictive Model

The Model

A predictive model taking MISD ratio, current frequency, and current temperature to predict the temperature 3 seconds into the future.

Going Granular: Mixed-Mode Kernels

- The 4 primary tiers establish the absolute boundary conditions (the thermal ceiling and floor).
- To map the continuous MISD-to-temperature transfer function, we build mixed-mode Firestarter kernels (e.g., 80/20, 60/40, 50/50, 20/80 ratios of FMLA to ADD).
- We will also use programmed duty-cycle variations (e.g., 80% density for 50ms, then 20% for 10ms) to train the model on transient changes seen in actual SLM workloads.

Part 3: Proposed LKM and EAS Integration

Loadable Kernel Module (LKM)

We will build an LKM that registers as a custom thermal_zone governor, taking over from step_wise.

Proactive EAS Integration

Instead of directly throttling frequency, the LKM feeds predicted thermal urgency into the Linux Energy Aware Scheduler (EAS) via arch_set_thermal_pressure().

Holistic Scheduling

EAS takes this "thermal pressure" constraint and autonomously handles task migration and frequency scaling—acting smoothly before the thermal limit is hit.

SECTION 4

Deliverables of the Project

The Four-Arm Experimental Design

A/B test running continuous SLM inference for 10 minutes, comparing four distinct control loops:

B0	B1	B2	B3
Default Linux	**DTS-Only Predictive**	**AMU-Only**	**MISD Governor**
Reactive status quo	PID Controller	Aggregate activity	Density-driven LKM

The Five Metrics We Will Collect

01	Average Tokens Per Second (TPS)	Measured via llama.cpp over 10 min. Proves sustained throughput improvements.
02	Prediction Lead Time	Time difference (expected 3–5 sec) between MISD indicating danger and sensor confirming.
03	False Positive Rate	Percentage of unnecessary throttling events triggered by the governor.
04	Thermal Violation Count	Number of times the device exceeds the safety threshold during the run.
05	Sensor Overhead	Performance cost (expected 2–5%) of running the PMU reading thread.

Final Showcase & Visualizations

The “Money Slide”

A time-series plot of Tokens Per Second (TPS) over 600 seconds across all four baselines, demonstrating how MISD avoids catastrophic throttling drops.

Summary Bar Charts

Visualizing Average TPS, False Positive Rate, and Prediction Lead Time across all arms.

The Ultimate Deliverable

Proven additive predictive value showing that MISD proactive intervention yields a measurable, stable performance gain for Edge AI workloads.

SECTION 5

Sustainability Impact

Why Proactive Thermal Governance is a Sustainability Win

Reactive throttling wastes energy. Proactive density-driven control eliminates that waste.

The Core Problem

Panic-throttling drops CPU frequency by 50–70%. During deep throttle, the system still draws significant baseline power but produces tokens at a fraction of normal rate. Energy-per-token is 3–5× worse. This sawtooth cycle repeats endlessly: ramp → hit wall → panic → cool → repeat.

The MISD Governor Fix

Small, preemptive frequency reductions keep the CPU operating continuously near (but safely below) the thermal limit. The CPU never enters the deep-throttle wasteland. Result: more tokens per joule.

Less Energy Per Token

Avoiding deep-throttle zones reduces total energy. At fleet scale (10K devices × 8 hrs/day), even 15% improvement saves thousands of kWh annually.

Higher Useful Work Ratio

Default: ~70% efficient runtime, ~30% wasted. MISD: 95%+ at efficient operating points. Near-zero time in energy-wasteful deep-throttle.

Simpler Cooling Hardware

Smoother thermal profiles mean smaller heatsinks, slower fans, or passive cooling — reducing material, weight, noise, and e-waste.

Evaluation: Metrics to Prove MISD is Better

A Three-Tier Framework: Computational Efficiency ⟶ Energy Efficiency ⟶ Sustainability Impact

Tier 1: Computational Efficiency

Metric	Default Governor	MISD Governor	Sustainability
Avg TPS (10-min)	Drops after thermal hit	Sustained near-peak; +15–20%	More work per unit time
TPS Stability (CV)	High variance (sawtooth)	Low variance (smooth)	Predictable edge latency
Throttle Events	Few but severe (−60% freq)	Many but gentle (−10% freq)	Less thermal cycling stress

Tier 2: Energy Efficiency (USB-C Power Meter)

Metric	Default Governor	MISD Governor	Sustainability
Energy Per Token (J/tok)	High — deep-throttle waste	~15% lower	Core sustainability metric
Wasted Energy	Significant during deep throttle	Near zero	Eliminates pure waste
Power Proportionality	Wild swings (TPS/Watt)	Flat, consistent ratio	Output scales with energy

Tier 3: Sustainability Impact (Projected)

Carbon / Million Tokens	Energy/token × grid intensity (India: ~700 gCO_2/kWh) ⟶ 15% less carbon
Annual Fleet Savings	10K devices × 8 hrs/day × 15% saving = thousands kWh/year
Cooling HW Reduction	Lower peak temps ⟶ smaller heatsink ⟶ enables passive cooling for edge
MISD Governor Overhead	Expected < 1% of system power (honest reporting)

Key Metric: Energy Per Token (J/tok) — directly maps to cost, carbon footprint, and battery life.